

INSTITUTO TECNOLÓGICO DE BUENOS AIRES

Maestría en Tecnología de la Información

Big Data

Primer Parcial: Diseño Preliminar de Arquitectura

Caso: Cloud Provider Analytics

Integrantes del Grupo 1:

María Mercedes Baron
Axel Marcelo Castro Benza
Lautaro Joaquín Farías
Nicolás Matías Kim
Andrés Podgorny

Fecha de Re-Entrega: 15 de junio de 2026

Profesor: Diego Mosquera

En esta re-entrega presentamos la arquitectura de datos corregida para el caso Cloud Provider Analytics. Decidimos dejar atrás la propuesta inicial basada en componentes heredados de Hadoop, que ya resultan obsoletos para este tipo de procesamiento. En su lugar, armamos un diseño moderno centrado en PySpark para el procesamiento y Apache Cassandra para el almacenamiento analítico, adaptándonos a los lineamientos y tecnologías sugeridos en la materia.

El propósito principal de este sistema es concentrar la telemetría de infraestructura y de negocio de la empresa de nube, lo que permitirá responder a consultas complejas de FinOps, el equipo de Soporte y la división de Producto.

1. El Caso de Uso y sus Necesidades

El principal desafío que enfrentamos en este proyecto es combinar fuentes de datos heterogéneas que ingresan con frecuencias y velocidades muy distintas. Por ejemplo, los eventos de telemetría de los servidores y servicios de nube llegan en un flujo continuo (streaming) y exigen un procesamiento veloz para detectar desvíos de costos o anomalías al instante. En cambio, otros datos como el perfil de los clientes (desde el CRM), las encuestas mensuales de satisfacción (NPS) o las conciliaciones de facturación son archivos estáticos que se actualizan de forma diaria o mensual. La solución que proponemos busca integrar ambos mundos sin perder precisión en el camino.

Justificación de Big Data (las 5V)

Dimensión (V)	Aplicación práctica en Cloud Analytics
Volumen	Al operar una nube global, recibimos millones de registros de cómputo y almacenamiento por día. Las bases de datos relacionales comunes colapsan frente a este volumen de inserciones.
Velocidad	La telemetría de consumo debe procesarse casi al instante. Usamos ventanas de tiempo para alertar a los clientes sobre picos de consumo inesperados antes de que llegue la factura.
Variedad	Trabajamos con formatos muy distintos: desde archivos CSV tabulares con datos de clientes y planes hasta series temporales semiestructuradas en formato JSONL para la telemetría.
Variabilidad	La estructura de los eventos no es fija. Con la evolución del esquema, pueden aparecer columnas nuevas (como uso de tokens de IA o huella de carbono) sin que el pipeline se caiga o falle.
Valor	Extraer insights limpios permite a la empresa optimizar gastos innecesarios (FinOps), evaluar cómo adoptan los usuarios las herramientas de IA generativa y auditar los acuerdos de nivel de servicio (SLA).

2. Diagrama de Arquitectura de Alto Nivel

Patrón elegido: Lambda

Para resolver la diferencia de tiempos de llegada y procesamiento de la información, elegimos una arquitectura tipo Lambda. Esta estructura nos permite separar el flujo de datos en dos caminos claros:

- **Capa batch (lotes):** Se encarga de procesar los datos de referencia que no cambian constantemente, como los clientes y los recursos del CRM, además de la facturación consolidada de cada mes y el historial de soporte. Al correr de forma periódica, asegura consistencia y exactitud.
- **Capa speed (streaming):** Procesa por micro-lotes las ráfagas continuas de telemetría para calcular costos diarios estimados. En esta rama usamos marcas de agua (watermarks) para dejar de lado cualquier evento que llegue demasiado retrasado.
- **Capa de serving (Cassandra):** Funciona como el punto de encuentro donde se consolidan los resultados de ambas ramas. Usamos Cassandra configurada bajo un esquema orientado a la consulta (query-first) para asegurar respuestas rápidas a los tableros de BI.

¿Por qué no elegimos Kappa? Si usáramos la arquitectura Kappa, tendríamos que tratar absolutamente todo (incluyendo el CRM y la facturación histórica) como si fueran flujos continuos de eventos en tiempo real. Esto exigiría reprocesar meses enteros de archivos estáticos pesados solo para corregir algún error, sumando una complejidad que no se justifica. Lambda nos da un aislamiento práctico: si el pipeline en tiempo real se interrumpe, no perdemos consistencia ni afectamos la facturación consolidada.

Armamos este diagrama para ilustrar de forma visual el recorrido de los datos y cómo interactúan las distintas capas y tecnologías en nuestra propuesta:

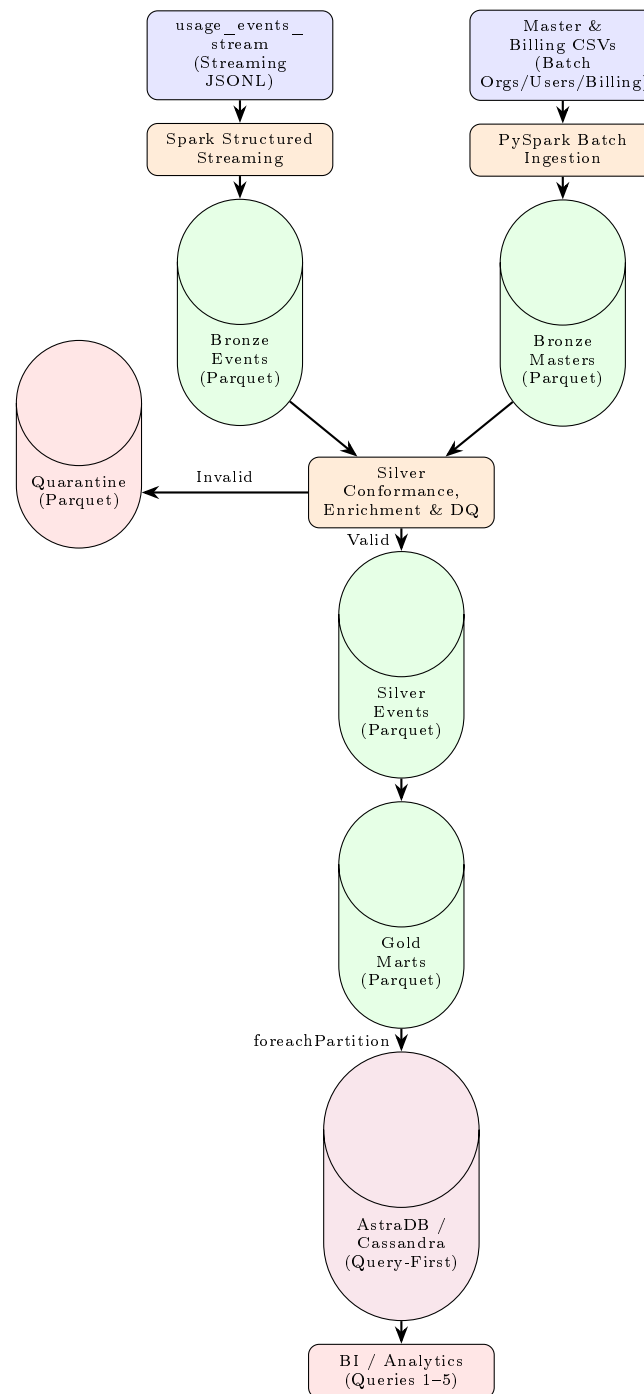


Figura 1: Diagrama conceptual de la arquitectura Lambda para Cloud Provider Analytics.

3. Mapeo de Requisitos a Componentes

Para conectar las necesidades de negocio con las herramientas técnicas, preparamos un mapeo detallado que justifica por qué elegimos cada componente del stack y cuál es su rol:

Requisito	Componente	5V	Justificación técnica
Almacenamiento escalable	Parquet (Bronze, Silver, Gold)	Volumen Variedad	La compresión columnar optimiza el espacio de almacenamiento. Particionar físicamente el disco por fecha y por servicio evita lecturas de archivos innecesarias y acelera las consultas.
Procesamiento streaming	Spark Structured Streaming	Velocidad Variabilidad	Lee archivos JSONL en micro-lotes, agrupa los consumos en ventanas de tiempo y usa marcas de agua para descartar cualquier dato demorado.
Procesamiento batch	PySpark Batch	Volumen Variedad	Une las dimensiones de clientes en archivos CSV y la facturación histórica. Realiza los cruces pesados y la agregación en memoria antes de guardar los resultados.
Capa de serving	AstraDB / Cassandra	Velocidad Valor	Motor NoSQL distribuido que, estructurado bajo el principio de query-first, responde las búsquedas de los tableros analíticos casi de inmediato.
Control de calidad	Quarantine (Parquet separado)	Variabilidad Valor	Una carpeta de aislamiento donde guardamos registros erróneos (como campos vacíos o valores negativos) para no interrumpir el procesamiento normal.
Evolución del esquema	Schema Enforcement en Spark	Variabilidad	Acepta sin problemas el paso de la versión 1 a la 2 del esquema (nuevas columnas de carbono e IA), completando los registros viejos con valores nulos.

4. Flujo de Datos (Data Pipeline)

En esta parte explicamos el camino que siguen los datos a través de las cuatro zonas del Data Lake hasta terminar consolidados en las tablas físicas de Cassandra:

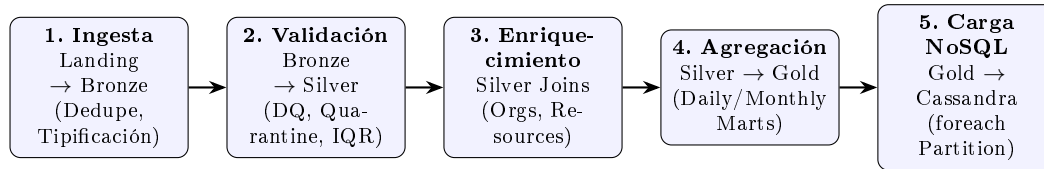


Figura 2: Etapas del pipeline de datos desde la ingesta hasta la capa NoSQL.

4.1 Zonas del Data Lake

1. **Landing (Datos crudos):** Es el punto de entrada al que llegan los archivos tal como son generados. Guardamos en formato inmutable los archivos CSV de clientes, el historial mensual de facturación y los archivos JSONL con la telemetría en tiempo real.
2. **Bronze (Estructura inicial):** Hacemos una primera lectura desde Landing y asignamos tipos de datos básicos a las columnas. También eliminamos registros duplicados según el identificador de evento e insertamos columnas para auditoría, como la fecha de procesamiento (`ingest_ts`) y el nombre del archivo de origen (`source_file`). Los archivos resultantes se guardan en Parquet.
3. **Silver (Calidad e Integración):** En esta etapa normalizamos los campos de texto, las fechas y los nombres de las regiones. Cruzamos la telemetría con los datos de clientes y recursos. Para lidiar con el cambio de versión del esquema (v1 y v2), completamos con nulos los campos faltantes de los registros antiguos. Además, calculamos los límites del **Rango Intercuartílico (IQR)** para cada servicio con el fin de etiquetar anomalías de costos y separamos cualquier dato sospechoso o mal formateado enviándolo a la zona de Quarantine.
4. **Gold (Modelos de negocio):** Construimos las agregaciones y tablas finales listas para consumo (totales de costos, recuentos de tickets de soporte y consumo de servicios de IA). Optimizamos la organización física en disco para facilitar y acelerar la transferencia de estos datos hacia Cassandra.

4.2 Modelado de la Capa de Serving (Marts en AstraDB/Cassandra)

Para garantizar que las búsquedas desde los tableros analíticos se resuelvan en milisegundos, modelamos las tablas de Cassandra aplicando la metodología query-first. Diseñamos claves primarias compuestas que nos permiten distribuir los registros de forma pareja entre los nodos del clúster (Partition Key) y ordenarlos internamente de forma eficiente (Clustering Columns):

Mart Lógico y Grano	Claves en Cassandra	Métricas y Atributos
FinOps Diario: org_daily_usage_by_service	PK: org_id CK: service, usage_date	Consumo de horas de CPU, volumen de almacenamiento en GB/h, cantidad de solicitudes, costos en dólares, tokens acumulados y huella de carbono estimada.
Facturación Mensual: revenue_by_org_month	PK: org_id CK: month	Suma de importes brutos, descuentos aplicados, impuestos calculados para el período y facturación neta consolidada en dólares.
FinOps Anomalías: cost_anomaly_mart	PK: org_id CK: service, usage_date	Costo registrado del día, flag de anomalía activado y los umbrales calculados mediante el Rango Intercuartílico.
Soporte Diario: tickets_by_org_date	PK: org_id CK: ticket_date	Volumen diario de casos, recuento de tickets de severidad alta, puntaje promedio de satisfacción del cliente y tasa de desvío en los tiempos de respuesta.
Producto / GenAI: genai_tokens_by_org_date	PK: org_id CK: usage_date	Volumen total de tokens consumidos por modelos fundacionales y costo estimado asociado para cada cliente.

Aclaración sobre la implementación física: Con el fin de ahorrar espacio y evitar escrituras duplicadas, agrupamos estos 5 modelos conceptuales en solo 3 tablas físicas dentro de Cassandra. Por ejemplo, los reportes de GenAI se obtienen filtrando por **service = 'genai'** en la tabla de uso diario general. Las anomalías de costo también pueden consultarse en esa misma tabla de uso o visualizarse directamente desde los archivos almacenados en la zona Gold de Parquet.

5. Supuestos y Riesgos Críticos del Proyecto

A continuación, enumeramos los supuestos en los que basamos el diseño y cómo planeamos atenuar los posibles problemas que puedan surgir:

ID	Tipo	Descripción del Riesgo / Supuesto	Estrategia de Mitigación / Validación
A1	Supuesto	La memoria RAM de los equipos de desarrollo es acotada para procesar millones de eventos en pruebas locales.	Optimizamos el tamaño de las particiones usando las funciones coalesce y repartition , evitando operaciones de shuffle global innecesarias.
A2	Supuesto	Se asume que los eventos retrasados en el stream no superan las 2 horas de demora desde su generación original.	Definimos marcas de agua con withWatermark en Spark, lo que permite cerrar ventanas de agregación y descartar registros muy demorados.
R1	Riesgo	Las caídas temporales de red podrían forzar reintentos de escritura y generar registros repetidos en la base de datos.	Activamos checkpoints persistentes en el flujo de Spark y definimos claves compuestas en Cassandra. Al funcionar como claves naturales, garantizamos actualizaciones idempotentes (upserts).
R2	Riesgo	El volumen de workers distribuidos en Spark podría abrir demasiadas conexiones concurrentes a Cassandra, saturando el servicio.	Implementamos la API foreachPartition en Spark para crear un solo grupo de conexiones por partición en cada ejecutor, en lugar de conectar por cada registro individual.
R3	Riesgo	La presencia de datos erróneos (como importes en negativo o picos de consumo atípicos) podría distorsionar los reportes financieros.	Implementamos reglas de validación en la transición entre las zonas Bronze y Silver para desviar cualquier registro sospechoso o incoherente a la zona de Quarantine.

6. Estimación de Esfuerzo y Recursos

Con el fin de estructurar el desarrollo del pipeline usando PySpark y Cassandra, definimos un equipo técnico compacto y planificamos el trabajo en base a etapas consecutivas de implementación:

6.1 Equipo propuesto

Rol	Responsabilidades principales en el proyecto
Data Engineers (x2)	Se ocuparán de desarrollar los flujos de datos batch y streaming en PySpark. También configurarán las marcas de agua, la lógica de eliminación de duplicados y las validaciones hacia la zona de Quarantine.
NoSQL Database Engineer (x1)	Estará a cargo del diseño físico y modelado query-first en Cassandra. Definirá las claves compuestas y optimizará las escrituras masivas concurrentes.
DevOps / Cloud Engineer (x1)	Administrará la infraestructura en la nube y el despliegue del entorno con Docker, garantizando la conectividad entre los clusters de Spark y Cassandra.
Cloud/FinOps Data Analyst (x1)	Validará que los datos en Cassandra sean correctos y creará las consultas SQL/CQL base para los paneles de control de soporte e informes de costos.

6.2 Fases de desarrollo estimadas

Fase	Actividades principales
1. Infraestructura y Landing	Montar el ambiente local, crear el Keyspace en AstraDB y configurar las carpetas iniciales para recibir las fuentes de datos sin modificaciones.
2. Ingesta (Bronze)	Programar el flujo de ingesta continuo para los logs JSONL y la carga batch de los archivos CSV maestros. Guardar los resultados en formato Parquet en la zona Bronze con tipos de datos iniciales.
3. Limpieza y Calidad (Silver)	Desarrollar las uniones (joins) entre fuentes, implementar las reglas de control de calidad con desvío a Quarantine y definir el cálculo de límites para detección de anomalías con el Rango Intercuartílico.
4. Agregación y Serving (Gold)	Crear las tablas agregadas finales en la zona Gold, implementar la escritura optimizada en paralelo hacia Cassandra y verificar que las consultas de negocio respondan en el tiempo esperado.